



**COMSATS University Islamabad,
Abbottabad Campus**

SOFTWARE DESIGN DESCRIPTION

(SDD DOCUMENT)

for

< KOJLO >
Version 1.0

By

Sayyam Tahir CIIT/SP23-BSE-014/ATD

Numaan Khan CIIT/SP23-BSE-012/ATD

Kazim Shauket CIIT/SP23-BSE-024/ATD

Supervisor

Muhammad Tariq Baloch

Bachelor of Science in Software Engineering (2023-2027)

Table of Contents

Revision History	3
1. Introduction.....	5
2. Design Methodology and software process model.....	6
3. System Overview	7
3.1 Architectural Design	7
3.2 Process Flow/Representation	10
4. Design Models [along with descriptions]	11
5. Data Design.....	15
5.1 Data Dictionary	16
6. Algorithm & Implementation	19
7. Software Requirements Traceability Matrix	24
8. Human Interface Design	24
8.1 Screen Images.....	26
8.2 Screen Objects and Actions.....	26
9. Appendix I	26

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
<Supervisor's Name>

Signature_____

Introduction

Khojlo is a mobile and web-based business discovery platform that aims to improve the visibility of newly established, affordable, and value-driven local businesses. The system provides users with a structured platform to search, discover, compare, and explore businesses while allowing business owners to manage verified business profiles. The project focuses on bridging the gap between customers and business owners by improving accessibility, business visibility, and informed decision-making.

The system currently covers the following functional modules:

1. **User Authentication and Profile Management**

- User registration and login
- User profile management
- Favorites and saved businesses

2. **Business Registration and Management**

- Business registration
- Business profile management
- Offers and promotions management

3. **Business Discovery Feed**

- Display newly opened businesses
- Display trending businesses
- Improve visibility of emerging businesses

4. **Search, Filtering, and Comparison System**

- Search businesses using keywords
- Filter businesses by category, location, and price
- Compare businesses based on pricing, services, and value

5. **Reviews and Ratings System**

- Submit reviews and ratings
- Display customer feedback
- Improve transparency and trust

6. **Maps and Location Integration**

- Display business locations on Google Maps
- Navigation assistance
- Location-based business discovery

7. **AI Chatbot Assistance Module**

- Answer business-related queries
- Retrieve information using a Retrieval-Augmented Generation (RAG) approach
- Provide responses based only on platform data

8. Admin and Moderation System

- Verify business profiles
- Moderate reviews and business listings
- Remove spam or inappropriate content

9. Chat and Messaging System

- Allow users to send direct messages to businesses
- Enable businesses to respond to customer inquiries
- Support one-to-one conversations
- Facilitate communication before visiting a business

The scope of the project is limited to business discovery, comparison, reviews, ratings, location-based services, and business information accessibility. Online ordering, payment processing, and delivery services are outside the scope of the system.

Design methodology and software process model

The Khojlo system follows the **Object-Oriented Design (OOD)** methodology. This methodology was selected because the system consists of multiple real-world entities such as users, business owners, businesses, reviews, ratings, offers, and administrators, which can be naturally represented as objects with their own attributes and behaviors.

Object-Oriented Design provides a modular structure where each component of the system can be developed, tested, and maintained independently. This improves code reusability, scalability, and maintainability while making future enhancements easier. Since the project is developed using Flutter for the frontend and FastAPI (Python) for the backend, both of which support object-oriented programming principles, the selected methodology is well suited to the overall system architecture.

The object-oriented approach also supports clear separation of responsibilities among system modules, making the overall design easier to understand and extend.

Software Process Model

The development of Khojlo follows the Agile Software Development process using the Scrum framework.

Agile was chosen because the project consists of multiple independent modules that can be developed incrementally. The development process is divided into iterative sprints, allowing individual modules to be designed, implemented, tested, and improved continuously throughout the project lifecycle.

The Scrum framework supports:

- Incremental development of project modules.
- Continuous testing and integration of new features.

- Regular progress reviews and team meetings.
- Efficient task distribution among team members.
- Flexibility to accommodate requirement changes during development.

Using Agile Scrum enables the development team to monitor progress effectively, identify issues early, and deliver functional components throughout the project instead of waiting until the end of development.

System overview

Khojlo is a cross-platform business discovery application that enables users to search, explore, compare, and interact with local businesses through both mobile and web platforms. The system primarily focuses on increasing the visibility of newly established and value-driven businesses while providing users with reliable business information in a structured and user-friendly environment.

The platform provides secure authentication for different user roles, including customers, business owners, and administrators. Business owners can register and manage verified business profiles, while users can discover businesses through search, filtering, comparison, reviews, ratings, interactive maps, and personalized discovery feeds. An AI-powered chatbot assists users by answering business-related questions using information stored within the platform. Additionally, the integrated messaging system allows users to communicate directly with business owners for inquiries, improving customer engagement and business accessibility.

The system adopts a client-server architecture in which the Flutter frontend communicates with a FastAPI backend through RESTful APIs. Business information, user accounts, reviews, messages, and other application data are securely stored in a PostgreSQL database. Google Maps services provide location-based search and navigation, while administrative tools ensure the authenticity of business listings and maintain overall platform quality through verification and content moderation.

The overall design emphasizes modularity, scalability, maintainability, and ease of future expansion while remaining focused on the primary objective of improving local business discovery and strengthening the connection between customers and business owners.

Architectural design

The Khojlo system follows a three-tier client-server architecture, which separates the application into the Presentation Layer, Application Layer, and Data Layer. This architecture promotes modularity, maintainability, scalability, and easier integration of external services. Each layer is

responsible for a specific set of tasks and communicates with adjacent layers through well-defined interfaces.

The **Presentation Layer** consists of the Flutter-based mobile and web applications through which users, business owners, and administrators interact with the system. This layer is responsible for displaying information, collecting user input, and communicating with the backend using RESTful APIs.

The **Application Layer** is implemented using FastAPI and contains the business logic of the system. It processes client requests, performs authentication and authorization, manages business operations, handles reviews, favorites, messaging, chatbot requests, and administrative functions. This layer also communicates with external services such as Google Maps APIs and the AI chatbot module whenever required.

The **Data Layer** uses PostgreSQL to store and manage all application data, including user accounts, business profiles, reviews, ratings, favorites, promotions, chat messages, and administrative records. The backend retrieves and updates information in the database whenever users interact with the system.

External services such as **Google Maps APIs** provide mapping, navigation, and location-based functionality, while the **RAG-based AI Chatbot** retrieves business information stored within the platform database to answer user queries.

This modular decomposition ensures that changes in one layer have minimal impact on other layers, making the system easier to develop, maintain, and extend in the future.

Module Relationships

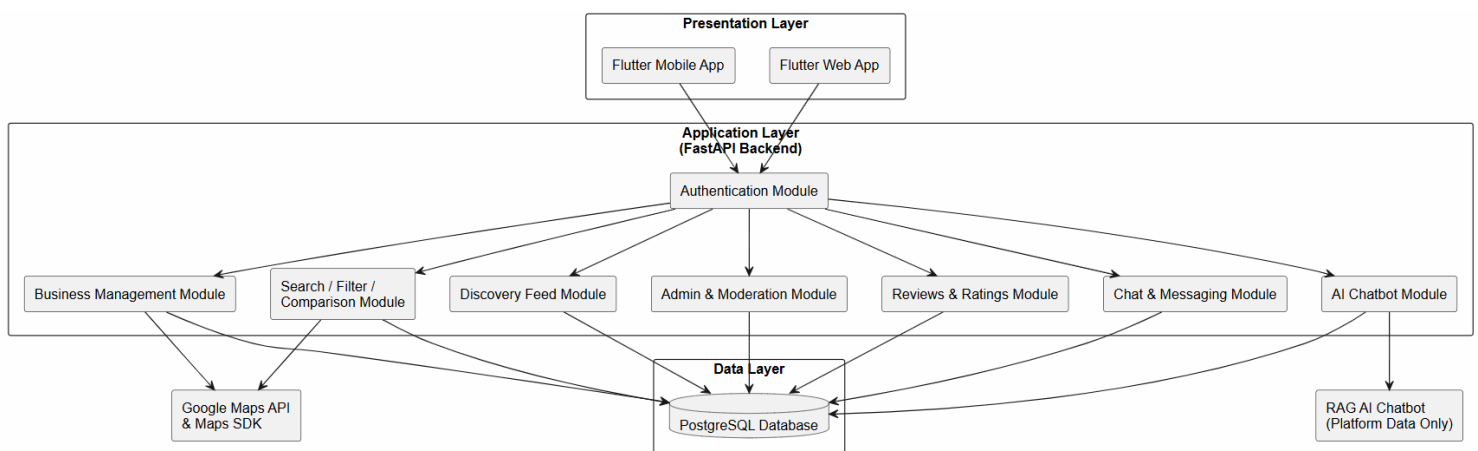
The system is divided into independent functional modules that collaborate to provide the overall functionality of Khojlo.

- **User Authentication Module** validates users before allowing access to protected features.
- **Business Management Module** allows business owners to register and manage business profiles.
- **Business Discovery Module** retrieves newly added and trending businesses from the database.
- **Search, Filter, and Comparison Module** processes user queries and retrieves relevant businesses.
- **Reviews and Ratings Module** stores and displays customer feedback.
- **Maps Integration Module** retrieves business locations using Google Maps services.
- **AI Chatbot Module** processes business-related questions and retrieves relevant information from the platform database using the RAG approach.

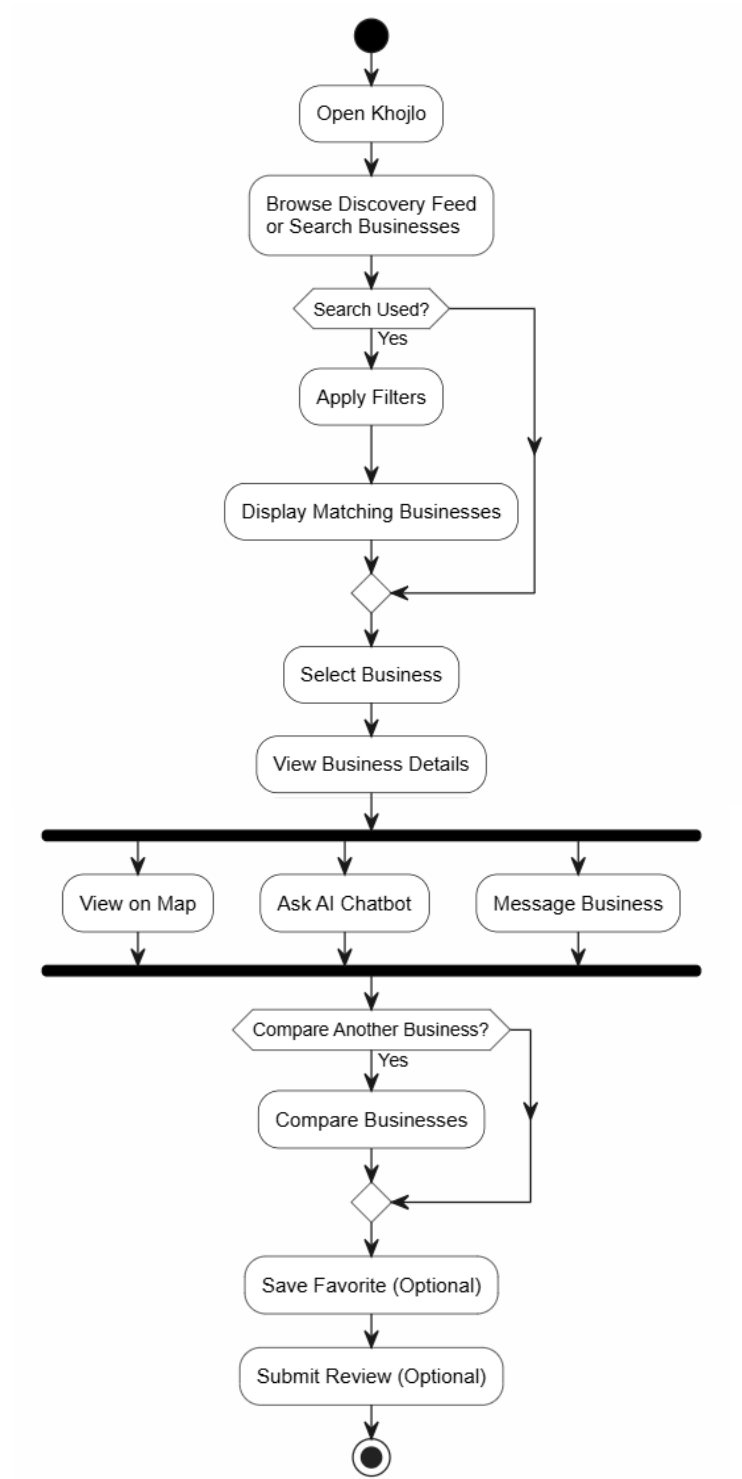
- **Chat and Messaging Module** enables direct communication between users and business owners.
- **Admin Module** verifies businesses, moderates reviews, and maintains platform integrity.

Although each module performs a separate responsibility, they communicate through the backend services and share information stored in the centralized PostgreSQL database. This modular organization reduces coupling between components while improving maintainability and future scalability.

Architecture Diagram



Activity Diagram

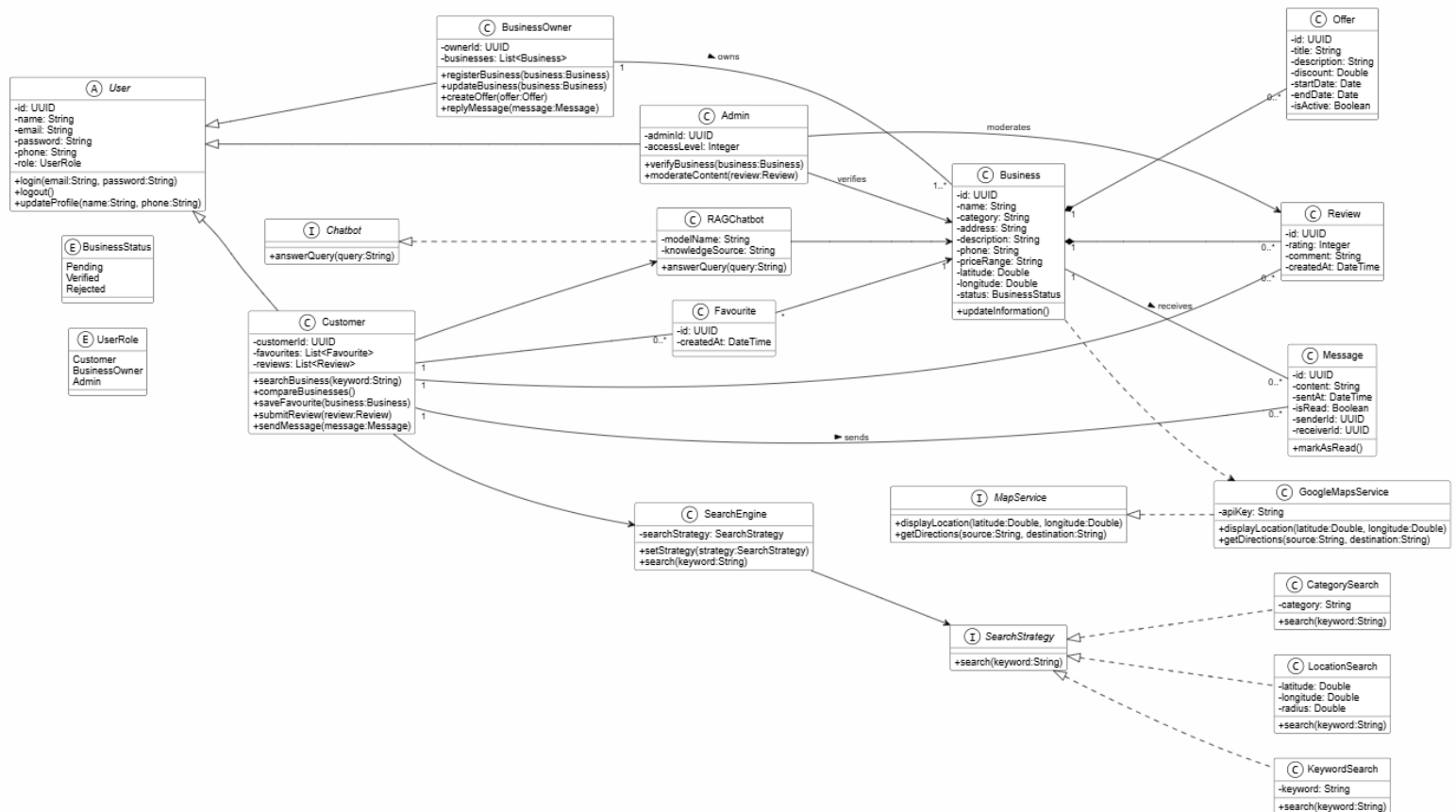


Design models

Class Diagram

Introduction

The class diagram represents the static structure of the Khojlo system by illustrating the major classes, their attributes, operations, and the relationships between them. It models the core entities of the application and demonstrates how object-oriented principles such as inheritance, association, composition, and interfaces are applied within the system.



Description

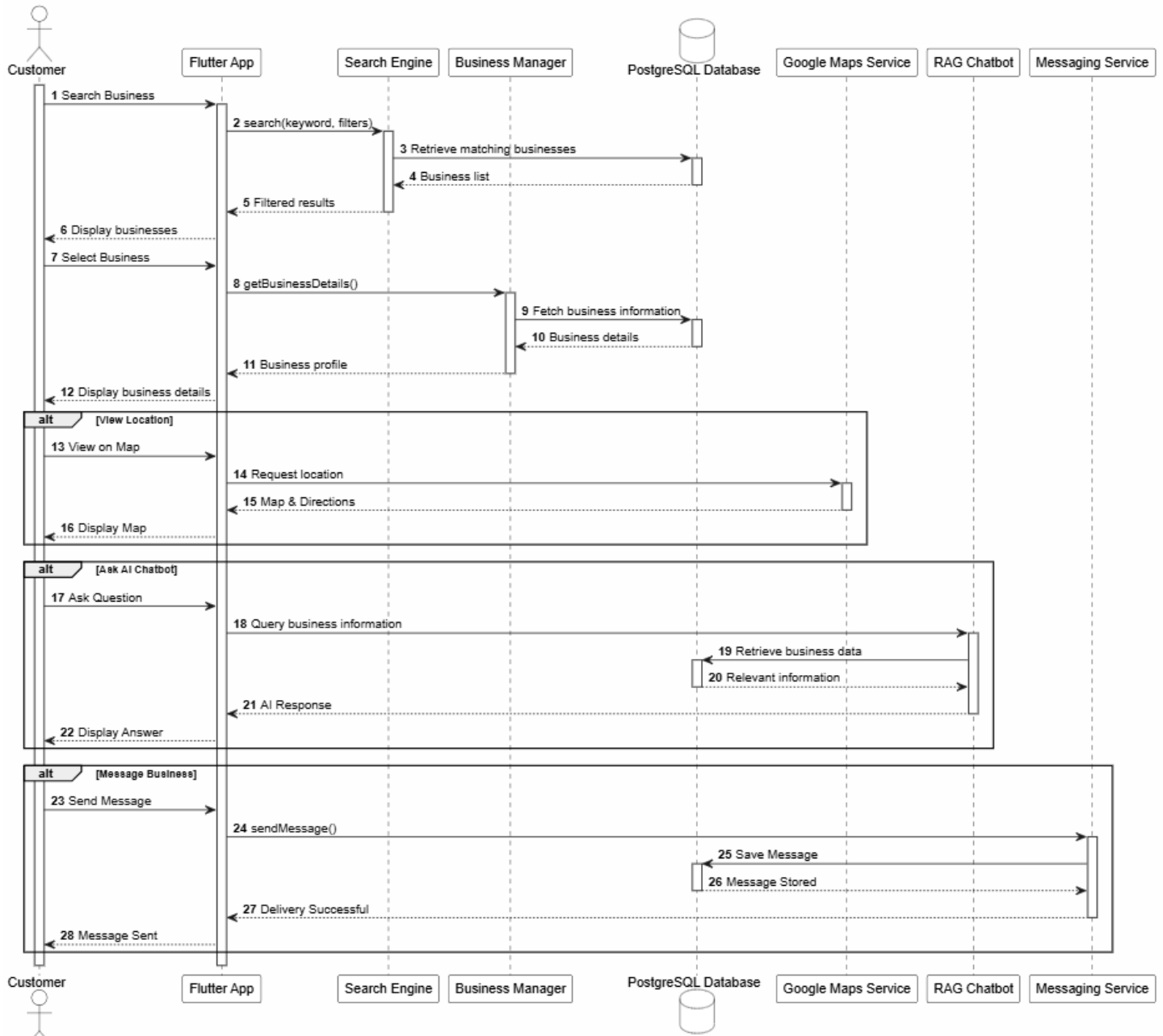
The class diagram is centered around the abstract **User** class, from which Customer, BusinessOwner, and Admin inherit common user attributes and operations. The BusinessOwner manages one or more Business entities, while each business may contain multiple Offers and Reviews. Customers can save businesses as favourites, submit reviews, and communicate directly with businesses through the Message class.

To support extensibility, the system uses the Strategy Pattern for implementing different search mechanisms through the SearchStrategy interface and its implementations. Similarly, the MapService and Chatbot interfaces abstract external services, allowing Google Maps and the RAG-based AI chatbot to be integrated without tightly coupling the application to a specific provider. This design improves modularity, maintainability, and future scalability.

Sequence Diagram

Introduction

The sequence diagram illustrates the interaction between the customer and the major software components of the Khojlo system during the business discovery process. It demonstrates how requests are processed by the application, how data is retrieved from the database, and how external services such as Google Maps and the AI chatbot are integrated to assist the user. The diagram emphasizes the chronological order of communication between the system components while performing one of the primary functionalities of the application.



Description

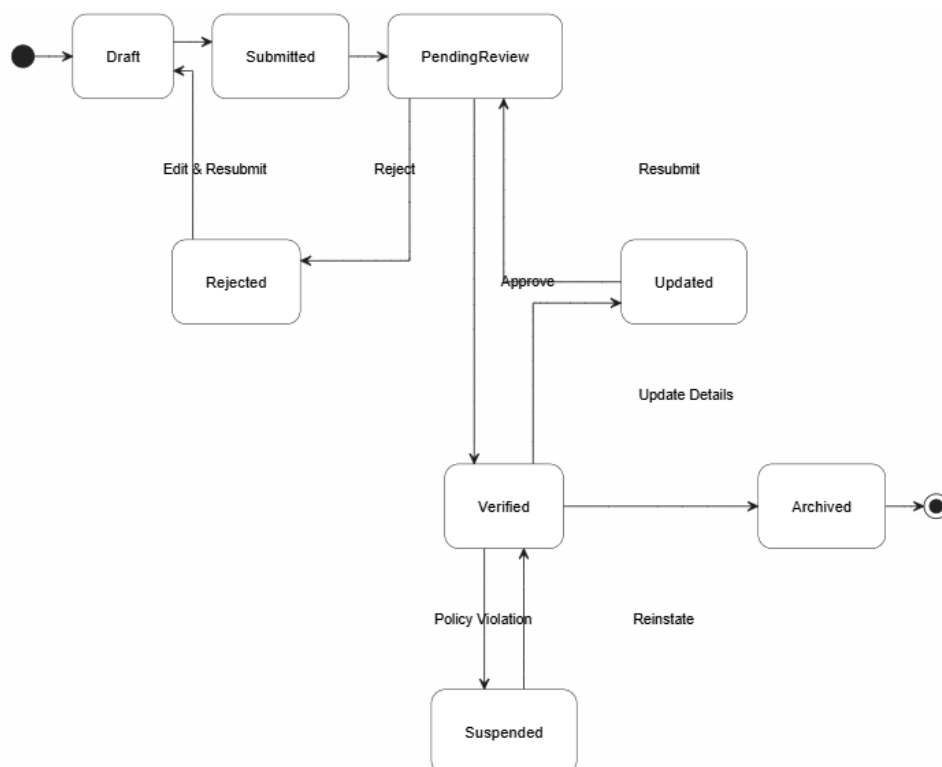
The sequence diagram represents the primary business discovery workflow of the Khojlo system. The process begins when the customer searches for a business through the Flutter application by providing keywords and optional filters. The search request is forwarded to the Search Engine, which retrieves the relevant business listings from the PostgreSQL database and returns the filtered results to the user interface.

After selecting a business, the application requests detailed information from the Business Manager, which retrieves the required data from the database and presents the complete business profile to the customer. From the business details page, the customer may perform additional actions depending on their requirements. The user can view the business location using the Google Maps Service, ask business-related questions through the RAG-based AI chatbot, or communicate directly with the business owner using the messaging service. Each optional interaction communicates with the appropriate subsystem before returning the requested information or confirmation to the user. This sequence demonstrates the collaboration between the application's internal components and external services to provide a comprehensive business discovery experience.

State Transition Diagram

Introduction

The state transition diagram illustrates the lifecycle of a business profile within the Khojlo system. It shows the different states a business passes through from registration until it becomes publicly available or is rejected. The diagram also represents the transitions triggered by business owners and administrators, ensuring that only verified businesses are visible to customers.



Description

The state transition diagram models the lifecycle of a business profile in the Khojlo platform. Initially, a business exists in the Draft state while the business owner enters the required information. Once the profile is submitted, it moves to the Submitted state and then enters Pending Review, where it awaits administrative verification.

If the administrator approves the profile, the business transitions to the Verified state, making it visible to customers on the platform. If the submitted information is incomplete or violates platform policies, the business enters the Rejected state. The business owner can update the rejected profile, returning it to the Draft state before resubmitting it for review.

A verified business may later update its information, causing it to enter the Updated state. To maintain the integrity of business listings, updated information is resubmitted for administrative review before becoming active again. Additionally, verified businesses may be temporarily moved to the Suspended state due to reports or policy violations and can later be reinstated after review. When a business permanently ceases operations or is removed from the platform, it transitions to the Archived state, marking the end of its lifecycle.

Data design

Data Design

The Khojlo system follows a **relational data model**, where application data is stored and managed using a **PostgreSQL** database. The information domain identified during the analysis phase is transformed into normalized relational tables representing the major entities of the system. Each entity corresponds to a real-world object such as users, businesses, reviews, offers, favourites, and messages, with relationships established through primary and foreign keys to ensure data consistency and integrity.

The system stores user information in a centralized **User** entity, from which different user roles (Customer, Business Owner, and Administrator) are managed through role-based authorization. Business owners can register and manage one or more business profiles, while each business maintains its own details, geographical location, operational information, promotional offers, customer reviews, and ratings.

Customers interact with the system by searching for businesses, saving favourite businesses, submitting reviews, communicating with business owners through direct messages, and requesting

information from the AI chatbot. These interactions generate transactional data that is stored separately to improve organization and scalability.

The application follows a client-server architecture where the Flutter frontend communicates with the FastAPI backend through RESTful APIs. The backend processes user requests, validates data, performs business logic, and interacts with the PostgreSQL database for data storage and retrieval. External services such as Google Maps are accessed through APIs and therefore do not maintain persistent application data. Similarly, the RAG-based AI chatbot retrieves relevant business information from the existing database instead of maintaining a separate application database.

The database design minimizes redundancy through normalization while maintaining efficient relationships among entities. This structure supports scalability, maintainability, and future enhancements without requiring significant changes to the existing database schema.

Database / Data Storage

The Khojlo system uses the following data storage components:

Database / Data Store	Purpose
PostgreSQL Database	Stores all persistent application data including users, businesses, reviews, offers, favourites, and messages.
Google Maps API	Provides location, map visualization, and navigation services. No application data is permanently stored.
RAG Knowledge Source	Retrieves business-related information from the platform database to generate AI responses. It does not maintain an independent transactional database for the application.

Data dictionary

Admin

Attributes

Attribute	Type	Description
id	UUID	Unique administrator identifier
name	String	Administrator name
email	String	Administrator email
password	String	Encrypted password

phone	String	Contact number
role	UserRole	User role

Methods

Method	Parameters	Description
verifyBusiness()	businessId	Verifies a business profile
moderateContent()	reviewId	Removes inappropriate content

Business Attributes

Attribute	Type	Description
id	UUID	Business identifier
name	String	Business name
category	String	Business category
address	String	Business address
description	String	Business description
phone	String	Contact number
priceRange	String	Price category
latitude	Double	Latitude coordinate
longitude	Double	Longitude coordinate
status	BusinessStatus	Verification status

Methods

Method	Parameters	Description
updateInformation()	businessDetails	Updates business information

BusinessOwner

Methods

Method	Parameters	Description
registerBusiness()	businessDetails	Registers a new business
updateBusiness()	businessId, businessDetails	Updates business information
createOffer()	offerDetails	Creates promotional offers
replyMessage()	messageId	Replies to customer messages

Customer

Methods

Method	Parameters	Description
searchBusiness()	keyword, filters	Searches businesses
compareBusinesses()	businessList	Compares businesses
saveFavourite()	businessId	Saves favourite business

submitReview()	businessId, rating, comment	Adds a review
sendMessage()	businessId, message	Sends message to business

Favourite Attributes

Attribute	Type	Description
id	UUID	Favourite identifier
createdAt	DateTime	Creation date

Message Attributes

Attribute	Type	Description
id	UUID	Message identifier
content	Text	Message body
sentAt	DateTime	Sending time
isRead	Boolean	Read status

Offer Attributes

Attribute	Type	Description
id	UUID	Offer identifier
title	String	Offer title
description	String	Offer description
discount	Decimal	Discount value
startDate	Date	Offer start date
endDate	Date	Offer expiry date

Review Attributes

Attribute	Type	Description
id	UUID	Review identifier
rating	Integer	Rating value
comment	Text	Customer review
createdAt	DateTime	Review submission date

User Attributes

Attribute	Type	Description
id	UUID	Unique user identifier
name	String	Full name
email	String	Email address
password	String	Encrypted password

phone	String	Contact number
role	UserRole	User type

Methods

Method	Parameters	Description
login()	email, password	Authenticates user
logout()	—	Ends current session
updateProfile()	profileData	Updates user profile

Algorithm & Implementation

Algorithm 1: User.login()

Description: Authenticates a registered user before granting access to the system.

Pseudo Code

BEGIN Login

INPUT email

INPUT password

IF email exists THEN

 Retrieve user record

 IF password matches THEN

 Create user session

 Redirect to dashboard

 ELSE

 Display "Invalid Password"

 ENDIF

ELSE

 Display "User Not Found"

ENDIF

END

Algorithm 2: Customer.searchBusiness()

Description: Searches businesses according to the entered keyword and selected filters.

```
BEGIN SearchBusiness

INPUT keyword
INPUT filters

Retrieve all verified businesses

FOR each business

    IF business matches keyword
      AND satisfies filters

      Add business to result list

    ENDIF

ENDFOR

Sort results

Display businesses

END
```

Algorithm 3: Customer.compareBusinesses()

```
BEGIN CompareBusinesses

INPUT selectedBusinesses

Retrieve details of each business

Compare

    Price
    Rating
    Services
    Offers
    Distance

Display comparison table

END
```

Algorithm 4: BusinessOwner.registerBusiness()

```
BEGIN RegisterBusiness
```

```
INPUT business information
Validate information
IF validation successful THEN
    Save business
    Set status = Pending
    Notify Administrator
ELSE
    Display validation errors
ENDIF
END
```

Algorithm 5: BusinessOwner.updateBusiness()

```
BEGIN UpdateBusiness
Select business
Modify information
Validate data
Save changes
IF verification required THEN
    Set status = Pending Review
ENDIF
Display success message
END
```

Algorithm 6: BusinessOwner.createOffer()

```
BEGIN CreateOffer
INPUT offer details
```

Validate dates
Save offer
Associate offer with business
Display confirmation
END

Algorithm 7: Customer.submitReview()

BEGIN SubmitReview

INPUT rating
INPUT comment
Validate review
Store review
Recalculate average rating
Update business rating
END

Algorithm 8: Customer.sendMessage()

BEGIN SendMessage

INPUT message
Validate message
Store message
Notify business owner
Display success
END

Algorithm 9: Admin.verifyBusiness()

BEGIN VerifyBusiness

Retrieve pending business

```
Review business details
IF requirements satisfied THEN
    Status = Verified
ELSE
    Status = Rejected
ENDIF
Notify business owner
END
```

Algorithm 10: Admin.moderateContent()

```
BEGIN ModerateContent
```

```
Retrieve reported content
```

```
Review content
```

```
IF violates policy THEN
```

```
    Remove content
```

```
ELSE
```

```
    Keep content
```

```
ENDIF
```

```
END
```

Algorithm 11: SearchEngine.search()

```
BEGIN Search
```

```
Receive search strategy
```

```
Execute strategy
```

```
Retrieve matching businesses
```

```
Return results
```

END

Algorithm 12: GoogleMapsService.displayLocation()

BEGIN DisplayLocation

Receive coordinates

Request Google Maps API

Receive map

Display location

END

Algorithm 13: RAGChatbot.answerQuery()

BEGIN AnswerQuery

Receive user query

Retrieve relevant business information

Generate contextual response

Return response

END

Software requirements traceability matrix

Req. No.	Requirement	Ref. Item	Design Component	Component Item(s)
FR01	User Registration and Login	Class Diagram	User	login(), updateProfile()
FR02	User Profile Management	Class Diagram	User	updateProfile()
FR03	Business Registration	Class Diagram	BusinessOwner	registerBusiness()
FR04	Business Profile Management	Class Diagram	BusinessOwner, Business	updateBusiness(), updateInformation()
FR05	Business Discovery Feed	Sequence Diagram	Business Discovery Process	Search Business Workflow

FR06	Search Businesses	Class Diagram	Customer, SearchEngine	searchBusiness(), search()
FR07	Filter Businesses	Class Diagram	SearchEngine, SearchStrategy	search(), apply search strategy
FR08	Compare Businesses	Class Diagram	Customer	compareBusinesses()
FR09	View Business Details	Sequence Diagram	Business Discovery Process	View Business Details
FR10	View Business Location	Class Diagram	GoogleMapsService	displayLocation()
FR11	Submit Reviews and Ratings	Class Diagram	Customer, Review	submitReview()
FR12	Save Favourite Businesses	Class Diagram	Customer, Favourite	saveFavourite()
FR13	Business Offers and Promotions	Class Diagram	BusinessOwner, Offer	createOffer()
FR14	AI Chatbot Assistance	Class Diagram	RAGChatbot	answerQuery()
FR15	Direct Messaging	Class Diagram	Customer, Message	sendMessage()
FR16	Business Reply to Messages	Class Diagram	BusinessOwner, Message	replyMessage()
FR17	Business Verification	State Transition Diagram	Business Lifecycle	Pending → Verified
FR18	Review Moderation	Class Diagram	Admin	moderateContent()
FR19	Business Moderation	Class Diagram	Admin	verifyBusiness()
FR20	Business Status Management	State Transition Diagram	Business Lifecycle	Draft, Pending, Verified, Rejected, Suspended, Archived

Human interface design

Describe the functionality of the system from the user's perspective. Explain how the user will be able to use your system to complete all the expected features and the feedback information that will be displayed for the user.

Screen images

Display screenshots showing the interface from the user's perspective. These can be hand-drawn, or you can use an automated drawing tool. Just make them as accurate as possible. (Graph paper works well.)

8.2 Screen objects and actions

A discussion of screen objects and actions associated with those objects

Appendix I

- How to design using UML (OOP): For guidance please follow the instructions mentioned in the link: <http://agilemodeling.com/artifacts/>
- How and when to design ER diagrams: For guidance please follow the instructions mentioned in the link: http://people.inf.elte.hu/nikovits/DB2/Ullman_The_Complete_Book.pdf
- Data flow diagrams: For guidance please follow the instructions mentioned in the link and book:
 - <http://www.agilemodeling.com/artifacts/dataFlowDiagram.htm>
 - Software Engineering –A Practitioner's approach by Roger Pressman
- Architecture diagram: For guidance please follow the instructions mentioned in the link and book:
 - Ian Sommerville – Software Engineering 9th Edition– Chapter 6